Universidad Nacional de Loja

Clase 3 – Sistemas Digitales

María del Cisne Valarezo Román

Arquitectura, Ciclo de Instrucción y Manipulación de Bits

1. El Ciclo de Instrucción de la CPU:

Toda CPU ejecuta un flujo continuo compuesto por tres fases principales:

- **Fetch (Buscar):** La CPU lee la instrucción guardada en la memoria (Flash) a través del bus.
- **Decode (Decodificar):** El decodificador traduce los bits de la instrucción para saber qué operación debe realizar la Unidad Aritmético-Lógica (ALU).
- **Execute (Ejecutar):** La CPU procesa el dato o realiza el cambio en los registros.

2. Ciclos de Lectura y Escritura en Memoria

- **Operación de Lectura (Read):**
    - La CPU coloca la dirección deseada (por ejemplo, 0x25) en el **Bus de Direcciones**.
    - Activa la señal de control **"Leer"** en el **Bus de Control**.
    - La memoria coloca los datos guardados en esa dirección dentro del **Bus de Datos**.
    - La CPU captura el byte recibido del bus.
- **Operación de Escritura (Write):**
    - La CPU coloca la dirección (ej. 0x25) en el **Bus de Direcciones**.
    - Coloca el valor binario a escribir (ej. 0b001000) en el **Bus de Datos**.
    - Activa la señal de control **"Escribir"** en el **Bus de Control**.
    - La memoria reemplaza el contenido del byte almacenado.

3. Arquitecturas: Von Neumann vs. Harvard

- **Von Neumann:** Usa un único bus compartido para datos e instrucciones. Debe traer secuencialmente (por turnos) primero la instrucción y luego el dato.

- **Harvard:** Utiliza buses separados (uno para la memoria de programa y otro para la de datos). Permite leer una instrucción y transferir datos simultáneamente.

4. Jerarquía y Tipos de Memoria

- **Flash (No Volátil):** Almacena el código compilado del programa. Se graba al subir el programa desde el IDE (como un libro impreso). Soporta aproximadamente 10 000 ciclos de escritura.
- **SRAM (Volátil):** Guarda variables de ejecución rápida. Se borra al apagar el chip (como una pizarra). Las lecturas y escrituras en SRAM no tienen límite de degradación.
- **EEPROM (No Volátil):** Para datos pequeños persistentes (parámetros de calibración, contadores) que no deben perderse al apagar. Soporta hasta 100 000 ciclos de escritura. *(Nota: La lectura NO desgasta ninguna memoria; solo el proceso de borrar y escribir)*.

Comparativa de Hardware:

- **ATmega328P (Arduino Uno):** Flash: 32 KB | SRAM: 2 KB | EEPROM: 1 KB.
- **ESP32:** Flash: 4 MB | SRAM: 520 KB | EEPROM: No incluye dedicada (se emula sobre la Flash).
- **UART, I2C, SPI:** Protocolos de comunicación serie para conectar el microcontrolador con otros chips o sensores.

5. Manipulación de Bits a Nivel de Registro (Bitwise)

Las operaciones a nivel de bit permiten modificar un solo pin o parámetro sin cambiar los demás bits del registro.

- **Crear Máscaras (Desplazamiento a la izquierda):**

1 << n desplaza un 1 a la posición del índice n.

Ejemplo: 1 << 4 transforma 0000 0001 en 0010 0000.

- **Setear a 1 (Poner Bit n en 1):**

Utiliza la operación **OR (|)**.

$$REG \mid = (1 \ll n)$$

- **Limpiar a 0 (Poner Bit n en 0):**

Utiliza la operación **AND (&)** junto con la **NOT (~)** para invertir la máscara.

$$REG \ \& \ = \sim(1 \ll n)$$

- **Invertir/Toggle (Cambiar Bit n):**

Utiliza la operación **XOR (^)**.

REG ^= (1 << n)

- **Consultar/Evaluar Bit n:**

Aplica una máscara **AND (&)** y evalúa si el resultado es distinto de cero.

$$\boldsymbol{if}\ (\boldsymbol{REG}\ \&\ (\mathbf{1} \ll \boldsymbol{n}))$$